

Natural Language Processing

Complete Tutorial: Pre-processing, BoW, TF-IDF, Naïve Bayes, Evaluation

GARP RAI Exam Preparation

Slide 1: What is Natural Language Processing?

Definition:

- NLP is an aspect of machine learning concerned with understanding and analyzing human language
- Works with unstructured, free text format
- Also known as: content analysis, text mining, computational linguistics
- Enables computers to understand, interpret, and generate human language

Key Insight:

- Most information exists in unstructured text format
- NLP is one of the most exciting and fast-developing applications of machine learning
- Bridges the gap between human language and computer understanding

Core Challenge: Converting unstructured text into structured data that machines can analyze!

Slide 2: Why is NLP Challenging?

Key Challenges:

- ❶ **Ambiguity:** Same words have different meanings (polysemy)
 - Example: "bank" (financial institution) vs "bank" (river edge)
- ❷ **Context dependence:** Meaning depends on surrounding text
 - "That's just great!" - sincere or sarcastic?
- ❸ **Noise:** Spelling errors, abbreviations, colloquialisms
 - "u r gr8" vs "you are great"
- ❹ **Informal language:** Social media, text messages
- ❺ **Sarcasm and tone:** Hard for machines to detect

Impact: NLP may fail to fully capture document essence!

Major Financial Applications:

1 Fraud Detection

- SEC used NLP to detect accounting fraud
- HSBC uses NLP for fraud detection

2 Customer Service

- Call routing based on keyword recognition
- Bank of America chatbots

3 Sentiment Analysis

- Social media monitoring
- Market reaction assessment
- BlackRock sentiment analysis

4 Document Analysis

- Readability assessment (Fog index)
- Document similarity
- JPMorgan COiN platform

Key Benefit: Processes thousands of documents faster than humans!

Advantages of NLP Over Manual Reading:

1 Speed

- Processes thousands of documents quickly
- Enables real-time analysis

2 Completeness

- No scope to miss any aspects
- Provided they are built into the design

3 Consistency

- All documents considered identically
- No scope for biases or inconsistencies

Important Note:

- NLP may miss nuance that humans would catch
- Ambiguity remains challenging
- Requires careful design and validation

Remember: NLP = speed + completeness + consistency!

Primary Data Sources:

1 Corporate Disclosures

- Mandatory filings: 10-Q, 10-K
- Equity and bond issue prospectuses
- Formal, structured language

2 Social Media Posts

- Generalist: X (Twitter), Reddit
- Financial: Stocktwits, Seeking Alpha
- Informal, noisy, colloquial

3 News Wires

- Wall Street Journal
- Dow Jones newswire
- Formal, professional language

Data Collection Methods: API, web scraping with Python, PDF extraction

Definition:

- From Latin "corpus" meaning "body"
- Collection of all documents retrieved for analysis
- May include hundreds or thousands of documents

Key Characteristics:

- 1 All documents in the corpus are processed together
- 2 Vocabulary is built from the corpus
- 3 Analysis results apply to the entire corpus

Example:

- All 10-K filings of S&P 500 companies
- All tweets about a specific stock
- All customer service emails from a bank

Important: The corpus defines the scope of your NLP analysis!

COiN (Contract Intelligence) Platform:

- Automated review of legal documents
- Focus on commercial loan agreements

Challenges Addressed:

- 1 Manual review was time-consuming
- 2 Manual review was prone to errors
- 3 Extracting information from unstructured text

Outcomes:

- 1 Significantly reduced review time
- 2 Improved consistency in information extraction
- 3 Improved accuracy in information extraction

Key Insight: NLP models trained on annotated legal documents improve over time!

Financial Institutions Using NLP:

- ❶ **HSBC:** Fraud detection
 - Analyze patterns in financial transactions
- ❷ **Bank of America (BoA):** Customer service chatbots
 - Automated customer interaction
- ❸ **BlackRock:** Market sentiment analysis
 - Analyze news and social media for trading insights
- ❹ **JPMorgan Chase:** COiN platform for legal documents
- ❺ **SEC:** Accounting fraud detection (early adopter)

Common Theme: NLP is transforming financial services operations!

What is Document Similarity?

- Comparing two or more documents
- Measuring how similar their content is
- Using vector representations

Applications in Finance:

1 New Information Assessment

- Determine if news article provides new information
- Avoid redundant analysis

2 Disclosure Tracking

- Assess if tone of accounting disclosures has changed
- Monitor consistency in communications
- Detect significant changes

Key Benefit: Identifies novel information and tracks changes over time!

Fog Index for Readability:

- Measures mean number of words per sentence
- Measures proportion of "complex" words (more than two syllables)

Other Readability Measures:

- 1 Passive voice usage
- 2 Technical terminology
- 3 Plain English compliance

Why It Matters in Finance:

- Organizations commit to plain English
- Websites and materials should be accessible
- Customer communications must be understandable

Goal: Make financial communications accessible to all customers!

Why Pre-processing Matters:

- Raw text is messy and unstructured
- Machines need clean, standardized input
- Quality of pre-processing affects analysis quality

Initial Steps:

- 1 Extract text from PDFs or other formats
- 2 Remove HTML code and tags
- 3 Handle emojis (convert to text or numerical values)
- 4 Use tools like demoji module in Python

Data Quality Depends on Source:

- Newswires: Formal, error-free
- Social media: Spelling errors, colloquialisms, abbreviations

Remember: Garbage in = garbage out!

What is Tokenization?

- Separating a document into sentences
- Then splitting sentences into words
- Removing punctuation, spacing, special symbols
- Converting to lowercase

Two Types:

1 Sentence Tokenization

- Split at periods (.), question marks (?), exclamation marks (!)

2 Word Tokenization

- Split sentences into individual words
- Remove punctuation and special characters

Example: "Hello world!" → ["Hello", "world"] after tokenization!

What are Stop Words?

- Common words with little informational value
- Included to make sentences flow
- Examples: "has", "a", "the", "also", "and", "of"

Why Remove Them?

- 1 Add noise to analysis
- 2 Don't help distinguish documents
- 3 Increase vector size unnecessarily
- 4 Improve computational efficiency

Important Caveat:

- What constitutes a stop word varies by application
- Words with no value in one context may be useful in another
- Customization may be needed

Key Insight: Stop words are context-dependent!

What is Stemming?

- Reduces words to their core/root forms
- Cuts off prefixes and suffixes
- Crude, rule-based approach

Examples:

- 1 "disappointing" → "disappoint"
- 2 "disappointed" → "disappoint"
- 3 "running" → "run"
- 4 "jumping" → "jump"

Advantages:

- Fast
- Simple to implement
- Reduces vocabulary size

Limitation: Can lose nuance and degrees of positivity/negativity!

What is Lemmatization?

- Finds the dictionary form (lemma) of words
- Considers word meaning and part of speech
- More sophisticated than stemming

Examples:

- 1 "running" → "run" (lemma)
- 2 "better" → "good" (lemma)
- 3 "went" → "go" (lemma)

Advantages:

- More accurate than stemming
- Always produces real words
- Preserves meaning

Disadvantage: Slower and more computationally expensive!

Slide 16: Stemming vs Lemmatization Comparison

Stemming:

- **Method:** Rule-based, cuts prefixes/suffixes
- **Speed:** Very fast
- **Accuracy:** Less accurate
- **Output:** May create non-words
- **Use when:** Speed is critical

Lemmatization:

- **Method:** Uses vocabulary and morphological analysis
- **Speed:** Slower
- **Accuracy:** More accurate
- **Output:** Always produces real words
- **Use when:** Accuracy is critical

Tradeoff: Speed vs Accuracy!

What is POS Tagging?

- Labels each word with its grammatical category
- Common tags: noun, verb, adjective, proper noun, adverb
- Provides syntactic information

Why It's Useful:

1 Handles Ambiguity

- "Reading" (town) = proper noun
- "reading" (activity) = verb

2 Handles Case Sensitivity

- "Polish" (language) = proper noun
- "polish" (to shine) = verb

3 Improves Accuracy

- Identifies proper nouns before lowercasing
- Preserves important distinctions

Best Practice: Apply POS tagging before lowercasing!

Problems with Lowercasing:

- ❶ **Loss of Proper Noun Information**
 - "Reading" (town) vs "reading" (activity)
 - Names of people, places, firms become indistinguishable
- ❷ **Meaning Loss**
 - "Polish" (language) vs "polish" (to shine)
 - Important distinctions are lost
- ❸ **Misclassification**
 - Can lead to incorrect analysis
 - Reduces accuracy

Solution:

- Apply POS tagging before lowercasing
- Preserve proper noun information

Remember: Lowercasing can lose proper noun distinctions!

Problems with Removing Punctuation:

1 Loss of Question Context

- "Is the CEO performing well?" → may seem positive
- Without "?", it's incorrectly classified as positive

2 Loss of Exclamation Meaning

- May denote humor or sarcasm
- Tone indication is lost

3 Loss of Emphasis

- Exclamation marks indicate strong emotion
- Important for sentiment analysis

Solution:

- Consider punctuation separately
- Use as additional features
- Context matters!

Key Insight: Punctuation provides valuable context!

Vocabulary Creation Methods:

1 Exogenous Creation

- Created ahead of time
- Applied consistently to documents
- Fixed and controlled

2 Corpus-Based Creation

- Every distinct word in all documents
- Comprehensive and complete
- May be very large

Vocabulary Management:

- Remove very frequent words (top 5%)
- Remove very rare words
- Balance vocabulary size

Tradeoff: Exogenous = Controlled; Corpus-based = Complete!

What is Feature Extraction?

- Turning processed text into numeric vectors
- Also known as text representation
- Foundation for all NLP analysis

Why It's Needed:

- 1 Machine learning models need numeric inputs
- 2 Text must be converted to numbers
- 3 Enables statistical analysis

Common Methods:

- 1 Bag of Words (BoW)
- 2 TF-IDF
- 3 Word Embeddings (Word2Vec)

Key Concept: All texts become numbers!

What is Bag of Words?

- Treats each word distinctly and equally
- Position and order are ignored
- Counts occurrences in each document

Key Characteristics:

- 1 **Independence:** Each word considered separately
- 2 **Equal Treatment:** All words have same importance
- 3 **Simplification:** Removes complex language structure

Limitations:

- Loses context
- Ignores negation words
- Misses phrase meanings

Example: "I like dogs" and "I don't like dogs" are different!

Binary BoW:

- 1 if word appears in document
- 0 if word does not appear
- Presence/absence only

Example: [1, 0, 1, 0, ...]

Count BoW:

- Counts how many times each word appears
- Non-negative integers
- Frequency matters

Example: [2, 0, 1, 0, ...]

When to Use Each:

- **Binary:** When presence is enough
- **Count:** When frequency matters
- Count usually gives more information

Remember: Binary = presence/absence; Count = frequency!

What is DTM?

- Rows = documents
- Columns = terms/words
- Entries = word counts or frequencies
- Dimensions: $|W| \times D$

Characteristics:

- 1 **Sparse:** Many zeros
- 2 **Large:** Many columns (vocabulary size)
- 3 **Quantitative:** Enables analysis

Example:

- 1,000 documents, 10,000 word vocabulary
- Matrix: $1,000 \times 10,000$
- Most entries are zero

Key Insight: DTM makes text analyzable mathematically!

Why Normalize?

- Repeated words can skew analysis
- Similar to outliers in econometrics
- Need to equalize influence

L2 Normalization:

- Divide each element by L2 norm
- $||v||_2 = \sqrt{\sum x_i^2}$
- Scales elements between 0 and 1
- All documents become unit length

Example:

- Original: [2, 0, 1, 0, 1]
- L2 norm: $\sqrt{4 + 0 + 1 + 0 + 1} = \sqrt{6} \approx 2.45$
- Normalized: [0.82, 0, 0.41, 0, 0.41]

Goal: Make all document vectors comparable!

What is the Dictionary Approach?

- Pre-defined list of words sharing a common characteristic
- Used to assess sentiment or topic
- Most used way to analyze text in finance research

Common Application:

- 1 Count positive words
- 2 Count negative words
- 3 Calculate net sentiment = proportion positive - proportion negative

Example Dictionary Categories:

- Positive: "good", "great", "excellent"
- Negative: "bad", "poor", "terrible"
- Neutral: "the", "is", "a"

Key Concept: Simple word counting with pre-defined lists!

Sample Text: "Firm XYZ has just reported a year-on-year rise in earnings before tax of just 0.1%, disappointing investors, despite total sales growth in double-digits. The company also highlighted that the previous safety worries with the new release had been resolved, which should underpin future growth. An analyst expressed relief, suggesting that there had been concerns that the accidents would have led to a decline in the firm's share of this competitive market."

Positive Words:

- Rise, grow (twice), resolve, relief = 5 positives

Negative Words:

- Disappoint, worry, concern, decline = 4 negatives

Net Sentiment: $5 - 4 = +1$ (slightly positive)

Caveat: Counterfactual sentences complicate analysis!

What is Loughran-McDonald Dictionary?

- Developed specifically for financial text analysis
- Based on extensive examination of 10-K filings
- Categories: positive, negative, uncertain, litigious, strong modal, weak modal

Why It's Important:

- 1 General dictionaries misclassify financial text
- 2 Financial words have specific meanings
- 3 "liability" is not negative in financial context

Accuracy:

- Around 75% accuracy on Thomson Reuters news articles
- Widely used in finance research

Key Insight: Domain-specific dictionaries are more accurate!

Primary Advantages:

1 Transparency

- Word lists are explicit and clear
- Easy to understand what's being measured
- Results are interpretable

2 Ease of Implementation

- Simple word counting
- No training required
- Fast to apply

3 Comparability

- Same word lists across corpora
- Consistent measurement across studies
- Enables comparison

Best For: Quick, transparent, comparable sentiment analysis!

Key Limitations:

1 Quality Dependence

- Only as good as word lists
- Incomplete lists → poor accuracy
- Ambiguities → misclassification

2 Context Limitation

- Designed for formal documents
- Poor for social media
- Colloquial language not captured

3 Narrow Scope

- Limited subjects available
- Time-consuming to create new dictionaries

4 Negation Issues

- "Not good" should be negative
- "Not bad" should be positive

Remember: Dictionary approach has many limitations!

What are n-grams?

- $n = 1$: uni-gram (single word)
- $n = 2$: bi-gram (two words)
- $n = 3$: tri-gram (three words)
- Treats contiguous words as single unit

Example: "credit spreads narrowed today"

- Uni-grams: ["credit", "spreads", "narrowed", "today"]
- Bi-grams: ["credit spreads", "spreads narrowed", "narrowed today"]

Purpose:

- Capture phrase meanings
- Handle negation words
- Address context issues

Key Concept: n contiguous words as one unit!

Basic BoW Limitations:

① Loses Phrase Meaning

- "red herring" $\neq$ red + herring
- "San Diego" $\neq$ San + Diego
- "credit spread" $\neq$ credit + spread

② Negation Words

- "not good" $\neq$ not + good
- "nothing useful" $\neq$ nothing + useful

How n-grams Help:

- ① Preserve phrase meaning
- ② Handle negations correctly
- ③ Capture context
- ④ More accurate sentiment analysis

Example: "not good" as bi-gram captures negation!

The Negation Problem:

- "not good" → should be negative sentiment
- Basic BoW sees "not" and "good" separately
- Misses the negation effect

n-gram Solution:

- Bi-grams capture "not good" as one unit
- "not sufficient" as one unit
- "never great" as one unit
- Preserves negation meaning

Example: "The service is not good"

- Bi-grams: ["The service", "service is", "is not", "not good"]
- "not good" detected as negative phrase

Key Insight: n-grams pair negation + following word!

What is TF-IDF?

- Term Frequency - Inverse Document Frequency
- Assigns higher weights to important words
- Words frequent in document but rare in corpus = high score

Components:

- 1 **TF (Term Frequency):** How often word appears in document
- 2 **IDF (Inverse Document Frequency):** How rare the word is across documents
- 3 **TF-IDF:** $TF \times IDF = \text{word importance score}$

Intuition:

- Common words ("the", "this") get zero weight
- Rare but relevant words get high weight
- Better than simple word counts

Key Formula: $TF\text{-}IDF = TF \times IDF$!

Slide 35: Term Frequency (TF) Formula

TF Formula:

- $TF_{i,j} = \frac{w_{i,j}}{|v_j|}$
- $w_{i,j}$: number of times term i appears in document j
- $|v_j|$: total number of terms in document j
- Normalizes by document length

Why Normalize?

- Longer documents have more words
- Without normalization, longer documents dominate
- Corrects for document length

Example:

- Word appears 3 times in 100-word document
- $TF = 3/100 = 0.03$
- Word appears 3 times in 50-word document
- $TF = 3/50 = 0.06$ (higher importance)

Remember: $TF = \text{word count} / \text{total words in document!}$

Slide 36: Inverse Document Frequency (IDF) Formula

IDF Formula:

- $IDF_i = \ln(D / \sum_j d_{i,j})$
- D = total number of documents
- $\sum d_{i,j}$ = number of documents containing term i

Interpretation:

- Rare words: High IDF
- Common words: Low IDF
- Words in all documents: $IDF = 0$

Example:

- 100 documents, word appears in 10 documents
- $IDF = \ln(100/10) = \ln(10) = 2.30$ (high)
- Word appears in 90 documents
- $IDF = \ln(100/90) = \ln(1.11) = 0.10$ (low)

Key Insight: IDF rewards rare words!

Four Documents:

- ① d1 = "this disappointing stock let me down"
- ② d2 = "disappointing earnings, down for this stock"
- ③ d3 = "earnings down for this month"
- ④ d4 = "Fed hikes rates this month"

IDF Values:

Word	Documents	IDF
disappointing	2	0.69
down	3	0.29
earnings	2	0.69
fed	1	1.39
hikes	1	1.39
month	2	0.69
stock	2	0.69
this	4	0.00

Observation: "this" has zero IDF!

TF Calculation:

- d1 has 6 words: each $TF = 1/6 = 0.17$
- d3 has 5 words: each $TF = 1/5 = 0.20$

TF-IDF Scores:

Word	d1	d2	d3	d4
this	0.00	0.00	0.00	0.00
down	0.05	0.05	0.06	-
fed	-	-	-	0.28
hikes	-	-	-	0.28
month	-	-	0.14	0.14

Key Insights:

- "this" = 0 (appears in all documents)
- Rare words in short documents = highest scores

Conclusion: Rare words get highest TF-IDF!

Common TF Variations:

- 1 **Binary TF:** 1 if word appears, 0 otherwise
- 2 **Raw count:** $w_{i,j}$ (no normalization)
- 3 **Log normalization:** $\ln(1 + w_{i,j})$
- 4 **Normalized:** $w_{i,j} / |v_j|$ (standard)

Common IDF Variations:

- 1 **Smooth IDF:** $\ln((D+1)/(\sum d_{i,j}+1)) + 1$
- 2 **Standard IDF:** $\ln(D/\sum d_{i,j})$
- 3 **Probabilistic IDF:** $\log((D - \sum d_{i,j})/\sum d_{i,j})$

Why Variations Exist:

- Different applications need different emphasis
- Some handle document length better
- Some are more robust

Important: Different software may use different variations!

What TF-IDF Does:

- 1 **Weights rare words higher**
 - Words in few documents get high IDF
 - More discriminatory power
- 2 **Weights common words lower**
 - Words in all documents get low IDF
 - Reduced noise

When to Use TF-IDF:

- Document classification
- Information retrieval
- Keyword extraction
- Feature selection

Comparison:

- Simple word count: treats all words equally
- TF-IDF: distinguishes important from common words

Remember: TF-IDF = better feature representation!

What is Naïve Bayes?

- Classification algorithm based on Bayes' theorem
- Assumes features are independent (naïve assumption)
- Popular for text classification

Why "Naïve"?

- Assumes word independence
- In reality, words are not independent
- Still works well in practice

Advantages:

- 1 Simple and fast
- 2 Solid classification accuracy
- 3 Works well with high-dimensional data
- 4 Easy to implement

Key Insight: Despite unrealistic assumptions, it works well!

Bayes' Theorem:

$$P(c|d) = \frac{P(d|c) \times P(c)}{P(d)}$$

Components:

- ❶ **Posterior:** $P(c|d)$
 - Probability of class given document
 - What we want to calculate
- ❷ **Likelihood:** $P(d|c)$
 - Probability of document given class
 - Estimated from training data
- ❸ **Prior:** $P(c)$
 - Unconditional probability of class
 - Prior knowledge about class distribution
- ❹ **Evidence:** $P(d)$
 - Normalizing constant
 - Cancels across classes

Key Idea: Posterior = Likelihood $\times$ Prior!

Slide 43: Naïve Independence Assumption

Independence Assumption:

- $P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \times P(w_2|c) \times \dots$
- Words are conditionally independent given class

Why It's Naïve:

- In reality, words are not independent
- "good" and "great" often appear together
- "not" and "good" are related

Why It Works:

- Despite unrealistic assumption
- Works well in practice
- Simple and effective

Simplified Classification:

- $\hat{c} = \arg \max_c P(c) \times \prod P(w_i|c)$

Remember: Naïve = independence assumption!

Prior Probability Formula:

- $P(c_i) = \frac{n(c_i)}{N}$
- $n(c_i)$: number of documents labeled as class i
- N : total number of documents

Example - Customer Service:

- 4 good reviews
- 2 bad reviews
- 2 indifferent reviews
- Total: 8 reviews

Calculations:

- $P(\text{good}) = 4/8 = 0.5$
- $P(\text{bad}) = 2/8 = 0.25$
- $P(\text{indifferent}) = 2/8 = 0.25$

Key Insight: Prior = class count / total documents!

Likelihood Formula:

- $P(w_i | c_j) = \frac{\text{count of } w_i \text{ in class } c_j}{\text{total words in class } c_j}$

Example - Customer Service:

- Class "good": 4 documents
- Word 2 appears in 1 of 4 good documents
- $P(w_2 | \text{good}) = 1/4 = 0.25$

Conditional Probabilities Table:

Word	Good	Bad	Indifferent
w1=1	0/4	2/2	1/2
w2=1	1/4	2/2	2/2
w3=0	2/4	1/2	1/2
w4=0	0/4	1/2	1/2

Note: Zero probabilities are a problem!

Posterior Calculation:

- $P(c|d) \propto P(d|c) \times P(c)$
- Compute for each class
- Choose class with highest posterior

Example - Customer Service:

- New document: words 1 and 2 present; words 3 and 4 absent

Calculations:

- $P(\text{good}|\text{words}) = 0 \times 0.25 \times 0.5 \times 0 \times 0.5 = 0$
- $P(\text{bad}|\text{words}) = 1 \times 1 \times 0.5 \times 0.5 \times 0.25 = 0.0625$
- $P(\text{indifferent}|\text{words}) = 0.5 \times 1 \times 0.5 \times 0.5 \times 0.25 = 0.03125$

Result: "Bad" has highest posterior probability!

Slide 47: Maximum A Posteriori (MAP)

MAP Definition:

- $\hat{c} = \arg \max_{c \in C} P(c|d)$
- Choose class that maximizes posterior

Simplified Form:

- $\hat{c} = \arg \max_{c \in C} P(d|c) \times P(c)$
- $P(d)$ cancels out

Example:

- $P(bad|words) = 0.67$
- $P(indifferent|words) = 0.33$
- $P(good|words) = 0$

MAP Class: "Bad" (0.67 $\hat{}$ 0.33 $\hat{}$ 0)

Key Concept: MAP = class with highest posterior!

The Problem:

- Word doesn't appear in training data for a class
- $P(w|c) = 0$
- Product becomes zero: $\prod P(w_i|c) = 0$
- Class probability becomes zero

Example:

- No "good" reviews contain word "slow"
- New document has "slow" and otherwise positive words
- Probability of "good" = 0

Consequences:

- Overly draconian
- Unrealistic zero probabilities
- Can misclassify documents

Solution Needed: Smoothing!

What is Laplace Smoothing?

- Adds 1 to all counts
- Ensures all probabilities are non-zero

Formula:

- $P(w|c) = \frac{\text{count}(w,c)+1}{\text{total words in } c + |V|}$
- $|V|$ = vocabulary size

Example:

- Before: $P(w|c) = 0/4 = 0$
- After: $P(w|c) = 1/6 = 0.167$

Effect:

- Zero probabilities become positive
- All probabilities pushed toward uniform
- Larger λ = stronger smoothing

Key Concept: Laplace smoothing = add 1 to all counts!

Before Smoothing:

- $P(\text{good}|\text{words}) = 0$
- $P(\text{bad}|\text{words}) = 0.67$
- $P(\text{indifferent}|\text{words}) = 0.33$

After Laplace Smoothing ($\lambda = 1$):

- $P(\text{good}|\text{words}) = 0.029$ (increased from 0)
- $P(\text{bad}|\text{words}) = 0.583$ (decreased)
- $P(\text{indifferent}|\text{words}) = 0.388$ (increased slightly)

Key Observations:

- 1 Zero probabilities eliminated
- 2 Classification unchanged (still "bad")
- 3 More realistic probability estimates

Important: Smoothing may not change classification!

What is Underflow?

- Multiplying many probabilities < 1
- Product becomes extremely small
- Below computer precision
- Rounds to zero

Example:

- $0.5^{100} \approx 7.9 \times 10^{-31}$
- $0.9^{100} \approx 2.7 \times 10^{-5}$
- For long documents, product $\rightarrow 0$

Solution:

- Take logarithms
- $\log(P_1 \times P_2 \times \dots) = \log(P_1) + \log(P_2) + \dots$
- Products become sums
- Computationally stable

Key Insight: Use logs to avoid underflow!

Products to Sums:

- $\log(P_1 \times P_2) = \log(P_1) + \log(P_2)$
- Avoids multiplication of small numbers
- Computationally stable

Preserves Ordering:

- Log is monotonic
- Class with highest probability = class with highest log probability
- Results unchanged

Simplified MAP:

- $\hat{c} = \arg \max_c [\log(P(c)) + \sum \log(P(w_i|c))]$

Example:

- Instead of: $0.5 \times 0.25 \times 0.33$
- Use: $\log(0.5) + \log(0.25) + \log(0.33)$

The Problem:

- Words in test data not in training vocabulary
- No information about their distribution
- Cannot calculate $P(w|c)$

Solution:

- Ignore unknown words
- Remove from test document
- They cannot help with classification

Why Ignore Them?

- 1 No information about their distribution
- 2 Cannot calculate likelihood
- 3 Would cause problems

Best Practice:

- Build vocabulary from training data
- Only use known words for classification

Remember: Unknown words = no classification information!

Assumption 1: Independence

- Each word is independent of others
- $P(w_1, w_2, \dots, w_n | c) = \prod P(w_i | c)$
- Simplifies calculations

Assumption 2: Equal Weighting

- Each word has equal information value
- No word is more important than others
- After stop word removal

Why These Assumptions:

- Make problem tractable
- Work surprisingly well
- Simple and fast

Key Insight: Naïve = independence + equal weighting!

Generative Classifiers (Naïve Bayes):

- Model the joint distribution $P(d, c)$
- Learn how data is generated
- Can generate new examples
- Examples: Naïve Bayes, Hidden Markov Models

Discriminative Classifiers:

- Model the decision boundary $P(c|d)$ directly
- Only care about classification
- Examples: Logistic regression, SVM

Key Difference:

- Generative: Complete model of data
- Discriminative: Only classification boundary

Remember: Naïve Bayes = generative classifier!

Customer Service Example:

- 8 customer feedback forms
- 4 good, 2 bad, 2 indifferent
- 4 words: "slow", "inefficient", "helpful", "great"

Data Table:

Customer	1	2	3	4	5	6	7	8
Word1	0	1	0	0	0	0	1	1
Word2	1	1	0	0	1	0	1	1
Word3	0	1	1	0	1	1	0	0
Word4	1	0	1	1	1	1	0	1
Label	G	B	G	G	I	G	I	B

New Document: Words 1 and 2 present; words 3 and 4 absent!

Priors:

- $P(\text{good}) = 4/8 = 0.5$
- $P(\text{bad}) = 2/8 = 0.25$
- $P(\text{indifferent}) = 2/8 = 0.25$

Conditional Probabilities:

Word	Good	Bad	Indifferent
w1=1	0/4	2/2	1/2
w2=1	1/4	2/2	2/2
w3=0	2/4	1/2	1/2
w4=0	0/4	1/2	1/2

Posteriors (Without Smoothing):

- $P(\text{good}|\text{words}) = 0$
- $P(\text{bad}|\text{words}) = 0.67$
- $P(\text{indifferent}|\text{words}) = 0.33$

Classification: Bad!

Multi-class Extension:

- Natural extension of binary case
- Compute $P(c|d)$ for each class
- Select class with highest probability
- No limitation on number of classes

Example (3 classes):

- $P(good|d) = 0.029$
- $P(bad|d) = 0.583$
- $P(indifferent|d) = 0.388$
- Select "bad" (highest probability)

Applications:

- Sentiment: positive, negative, neutral
- Topic classification: finance, politics, sports
- Document categorization

Key Point: Choose class with highest posterior!

Simplicity:

- Easy to understand
- Easy to implement
- Few parameters to tune

Speed:

- Fast training
- Fast prediction
- Scales to large datasets

Accuracy:

- Solid classification accuracy
- Works well despite assumptions
- Competitive with more complex models

When to Use:

- High-dimensional data
- Text classification
- Quick baseline models

Summary: Simple, fast, and accurate!

Slide 60: Appendix Problem Walkthrough

Data:

- 5 buys, 5 sells
- Prior: $P(\text{buy}) = 0.5$, $P(\text{sell}) = 0.5$
- New document: words 1, 2, 3 present; words 4, 5 absent

Conditional Probabilities:

Word	$P(w=1 \text{buy})$	$P(w=1 \text{sell})$
1	4/5	1/5
2	5/5	2/5
3	2/5	3/5
4	1/5	3/5
5	2/5	4/5

Result:

- $P(\text{buy}|\text{words}) = 0.98$
- $P(\text{sell}|\text{words}) = 0.02$

Classification: Buy (98% probability)!

What are Word Embeddings?

- Represent words as dense vectors
- Capture semantic meaning
- Similar words have similar vectors
- Based on distributional hypothesis

Distributional Hypothesis:

- "You shall know a word by the company it keeps"
- Similar meanings appear in similar contexts
- Words appearing together are related

Advantages over BoW:

- Captures meaning
- Dense vectors (not sparse)
- Similar words close in space

Key Concept: Words as vectors in semantic space!

What is Word2Vec?

- Developed by Google (Mikolov et al., 2013)
- Neural network-based algorithm
- Generates word embeddings
- Reduces dimensionality

Two Architectures:

1 CBOW (Continuous Bag of Words)

- Predict target word from context
- Faster to train
- Better for frequent words

2 Skip-gram

- Predict context from target word
- Better for rare words
- More computationally expensive

Capability: "king - man + woman equals queen"!

Context Capture Methods:

① Position Information

- Track which words appear together
- Window-based co-occurrence

② Surrounding Words

- Consider words before and after
- Skip-gram uses surrounding context
- CBOW uses context to predict target

One-hot Limitation:

- Large and sparse
- No notion of similarity
- Each word independent

Embedding Advantage:

- Dense vectors
- Similar contexts = similar vectors

CBOW (Continuous Bag of Words):

- Input: context words
- Output: target word
- Faster to train
- Better for frequent words

Skip-gram:

- Input: target word
- Output: context words
- Better for rare words
- More computationally expensive

Choice Depends On:

- Data size
- Word frequency distribution
- Computational resources

Remember: Skip-gram = target $\rightarrow$ context; CBOW = context $\rightarrow$ target!

What is One-Hot Encoding?

- Vector length = vocabulary size
- One position = 1 (the word's position)
- All other positions = 0

Example:

- Vocabulary: ["apple", "banana", "orange"]
- "banana" = [0, 1, 0]

Limitations:

- Very sparse
- No semantic meaning
- All words equidistant
- Cannot capture relationships

Comparison with Embeddings:

- One-hot: Sparse, no meaning
- Embeddings: Dense, captures meaning

Key Insight: One-hot has no semantic information!

The Problem:

- Vocabulary can have 10,000+ words
- Vectors are very long
- Very sparse (many zeros)
- Slow and inefficient

Reduction Methods:

- 1 **PCA:** Principal Component Analysis
- 2 **SVD:** Singular Value Decomposition
- 3 **Word2Vec:** Neural network-based
- 4 **t-SNE:** For visualization

Benefits:

- Dense representations
- Faster computation
- Better performance
- Captures relationships

Goal: Sparse $\rightarrow$ Dense representations!

BoW Limitations:

- Loses context
- No semantic meaning
- Sparse vectors
- Ignores word relationships

Embedding Advantages:

1 Semantic Meaning

- Similar words close in space
- Captures relationships
- Analogies possible

2 Dense Vectors

- Lower dimensional
- More efficient
- Better for downstream tasks

3 Context Awareness

- Considers surrounding words
- Better representation

Key Difference: Embeddings capture meaning; BoW doesn't!

Skip-gram Architecture:

- Input: target word
- Output: context words (surrounding words)
- Uses neural network
- Predicts probability of context words

Training Objective:

- Maximize probability of context words
- Given target word
- Slide window over text
- Learn embeddings

Window Size:

- Typically 2-5 words each side
- Larger window = more global context
- Smaller window = more syntactic

Key Insight: Skip-gram learns from surrounding words!

Famous Analogy:

- $v_{king} - v_{man} + v_{woman} \approx v_{queen}$
- Captures gender relationship
- Preserves arithmetic operations

Other Examples:

- "Paris - France + Italy equals Rome"
- "walking - walk + run equals running"
- "sister - woman + man equals brother"

Why It Works:

- Vector space captures semantic relationships
- Similar relationships have similar vector offsets
- Enables analogical reasoning

Key Concept: Vector arithmetic captures meaning!

Evaluation Approaches:

1 Labeled Data

- Compare against known labels
- Use confusion matrix
- Compute accuracy, precision, recall, F1

2 Unlabeled Data

- Unsupervised evaluation methods
- Qualitative assessment
- Subject matter expert review

Other Considerations:

- Speed of algorithm
- Data requirements
- Computational resources

Important: Evaluation depends on available labels!

Confusion Matrix Structure:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Metrics:

- ① **Accuracy:** $(TP + TN)/(TP + TN + FP + FN)$
- ② **Precision:** $TP/(TP + FP)$
- ③ **Recall:** $TP/(TP + FN)$
- ④ **F1:** $2 \times (Precision \times Recall)/(Precision + Recall)$

Key Insight: Foundation for all classification metrics!

Accuracy Formula:

$$\bullet \text{ Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Interpretation:

- "Percentage of all predictions that were correct"
- Most intuitive classification metric
- Easy to communicate

Limitations:

- Doesn't distinguish between error types
- Misleading for imbalanced datasets
- Doesn't account for costs of errors

Example:

- 98% legitimate, 2% fraud
- Model predicts "legitimate" always
- Accuracy = 98% (seems great!)
- But catches zero fraud!

Warning: Accuracy can be misleading!

Precision Formula:

$$\bullet \text{ Precision} = \frac{TP}{TP+FP}$$

Managerial Question:

- "Of all the alerts our model generated, how many were actually real issues?"
- "If we act on this prediction, how often will we be right?"

When Precision Matters:

- High cost of investigation (AML alerts)
- Customer friction concerns
- Operational resource constraints

Example:

- 432 true positives, 121 false positives
- Precision = $432 / 553 = 78.1\%$

Key Insight: Precision = "of predicted positives, how many were right?"

Recall Formula:

- $Recall = \frac{TP}{TP+FN}$

Also Known As:

- Sensitivity
- True Positive Rate

Managerial Question:

- "Of all real issues that occurred, how many did we catch?"
- "What percentage of actual positives did we identify?"

When Recall Matters:

- Missing a positive has severe consequences
- Fraud detection, default prediction
- Compliance monitoring

Key Insight: Recall = "of actual positives, how many did we catch?"

F1 Score Formula:

$$\bullet \quad F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why Harmonic Mean?

- Harmonic mean penalizes imbalance
- Low precision or low recall $\rightarrow$ Low F1
- Better than arithmetic mean

Interpretation:

- Bounded between 0 and 1
- Close to 1 = high precision and high recall
- Low score = poor precision, poor recall, or both

Example:

- Precision = 78.1%, Recall = 72.0%
- $F1 = 2 \times (0.781 \times 0.720) / (0.781 + 0.720) = 74.9\%$

Key Concept: F1 balances precision and recall!

Labeled Data Evaluation:

- Documents have known labels
- Compare machine classifications with labels
- Use confusion matrix metrics
- Accuracy, Precision, Recall, F1

Unlabeled Data Evaluation:

- No "right answer" to compare
- Need different evaluation approaches

Methods for Unlabeled Data:

- 1 **Subject Matter Experts**
 - Manually review sample
 - Assess accuracy qualitatively
- 2 **Unsupervised Metrics**
 - Cohesion and separation
 - Silhouette score
- 3 **Indirect Validation**
 - Check if results make sense
 - Correlation with known variables

Key Insight: Unlabeled data requires alternative evaluation!

Practical Considerations:

1 Data Volume

- NLP datasets can be enormous
- Millions of documents
- Billions of words

2 Sparsity

- Sparse vectors are inefficient
- Slow to process
- Memory intensive

3 Model Complexity

- Neural networks are slow to train
- Require large amounts of data
- May be impractical

Tradeoffs:

- Accuracy vs speed
- Complexity vs feasibility
- Resource constraints matter

Remember: NLP can be computationally expensive!

Loss of Context:

- Word order ignored
- Phrase meaning lost
- No understanding of relationships

No Semantic Meaning:

- Words independent
- No concept of similarity
- "good" and "great" treated equally

Negation Issues:

- "not good" vs "good" both contain "good"
- Sentiment is reversed but BoW doesn't know

Advanced Solutions:

- n-grams
- Skip-grams
- Word embeddings
- Contextual embeddings (BERT)

Key Insight: BoW fails to capture context and meaning!

n-grams:

- n contiguous words as one unit
- Captures phrase meaning
- Handles negations

Skip-grams:

- Predict context from target
- Captures semantic relationships
- Word2Vec uses this

Word Embeddings:

- Dense vector representations
- Similar words close together
- Captures meaning

Contextual Embeddings:

- BERT, GPT, etc.
- Context-dependent representations
- State-of-the-art

Evolution: BoW → n-grams → Embeddings → Contextual!

Key Challenges:

1 Ambiguity

- Same words have different meanings
- Polysemy and homonymy

2 Context Dependence

- Meaning depends on surrounding words
- Need to capture relationships

3 Noise

- Spelling errors
- Abbreviations
- Colloquialisms

4 Tone Detection

- Sarcasm hard to detect
- Sentiment nuances

Remember: NLP is challenging but improving rapidly!

NLP Pipeline Steps:

1 Data Collection

- API, web scraping, PDF extraction
- Build corpus

2 Pre-processing

- Tokenization
- Stop word removal
- Stemming/Lemmatization

3 Feature Extraction

- BoW, TF-IDF, Embeddings
- Document Term Matrix

4 Modeling

- Dictionary-based or Machine Learning
- Naïve Bayes, SVM, Neural Networks

5 Evaluation

- Confusion Matrix
- Accuracy, Precision, Recall, F1

Key Concept: End-to-end process from text to insights!

Key Steps:

1 Tokenization

- Sentence tokenization
- Word tokenization

2 Stop Word Removal

- Remove common words
- Context-dependent

3 Stemming

- Fast, less accurate
- Rule-based

4 Lemmatization

- Slower, more accurate
- Uses dictionary

5 Feature Extraction

- Convert to vectors

Remember: Quality pre-processing is essential!

Methods:

1 Bag of Words (BoW)

- Binary or Count
- Ignores order
- Simple but limited

2 TF-IDF

- $TF \times IDF$
- Weights rare words higher
- Better than simple counts

3 Word Embeddings

- Dense vectors
- Captures meaning
- Word2Vec, GloVe

Evolution: BoW $\rightarrow$ TF-IDF $\rightarrow$ Embeddings!

Key Points:

1 Definition

- Pre-defined word lists
- Share common characteristics

2 Advantages

- Transparent
- Easy and fast
- Comparable

3 Limitations

- Only as good as word lists
- Formal documents only
- Narrow scope
- Negation issues

Example: Loughran-McDonald financial dictionary (75% accuracy)

Best For: Quick, transparent sentiment analysis!

Key Points:

1 Definition

- n contiguous words as one unit
- Uni-grams, bi-grams, tri-grams

2 Why Use Them

- Capture phrase meaning
- Handle negations
- Address context issues

3 Example

- "not good" as bi-gram
- "credit spreads" as bi-gram

BoN (Bag of n-grams):

- BoW applied to n-grams
- More context-aware than BoW

Key Insight: n-grams capture phrase-level meaning!

Key Points:

1 TF = Term Frequency

- $TF_{i,j} = \frac{w_{i,j}}{|v_j|}$
- Word count / total words

2 IDF = Inverse Document Frequency

- $IDF_i = \ln(D / \sum d_{i,j})$
- Rewards rare words

3 TF-IDF = TF $\times$ IDF

- Higher weight for rare, frequent words
- Zero for words in all documents

Use Cases:

- Document classification
- Information retrieval
- Keyword extraction

Key Insight: TF-IDF identifies important words!

Key Points:

1 Based on Bayes' Theorem

- $P(c|d) \propto P(d|c) \times P(c)$

2 Naïve Assumptions

- Independence of words
- Equal weighting of words

3 Pros

- Simple and fast
- Solid accuracy
- Works with high-dimensional data

4 Cons

- Unrealistic assumptions
- Zero probability problem
- Requires smoothing

Best For: Text classification, quick baselines!

Why Smoothing is Needed:

- Zero probability problem
- Word missing from training class
- Unrealistic zero probabilities

Laplace Smoothing:

- Add 1 to all counts
- $$P(w|c) = \frac{\text{count}(w,c)+1}{\text{total words in } c+|V|}$$
- Ensures all probabilities > 0

Effects:

- Zero probabilities eliminated
- Pushed toward uniform distribution
- Similar to regularization

Remember: $\lambda = 1$ is most common (Laplace smoothing)!

Key Points:

1 Definition

- Dense vector representations
- Capture semantic meaning

2 Word2Vec

- Google-developed algorithm
- CBOW or Skip-gram

3 Advantages

- Captures meaning
- Dense vectors
- Semantic relationships

4 Limitations

- Requires large corpus
- Computationally expensive

Key Insight: Embeddings enable "king - man + woman equals queen"!

Key Points:

1 Labeled Data

- Confusion Matrix
- Accuracy, Precision, Recall, F1

2 Unlabeled Data

- Subject matter experts
- Unsupervised metrics
- Indirect validation

3 Other Considerations

- Speed of algorithm
- Data requirements
- Resource constraints

Remember:

- Evaluation depends on available labels
- Multiple metrics provide complete picture
- Speed and resources matter

Key Insight: Choose evaluation method based on data and needs!

Key Applications:

1 Fraud Detection

- SEC, HSBC
- Analyze financial reports

2 Customer Service

- Call routing
- Chatbots (BoA)

3 Sentiment Analysis

- Social media monitoring
- BlackRock

4 Document Analysis

- Legal document review (JPMorgan COiN)
- Readability assessment
- Document similarity

Key Insight: NLP is transforming financial services!

Pitfall 1: Ignoring Context

- BoW loses meaning
- Solution: n-grams, embeddings

Pitfall 2: Negation Handling

- "not good" misclassified
- Solution: n-grams

Pitfall 3: Case Sensitivity

- "Reading" vs "reading"
- Solution: POS tagging

Pitfall 4: Over-smoothing

- Too much smoothing loses signal
- Solution: Balance λ

Remember: Consider context and assumptions carefully!

When to Use Dictionary Approach:

- Formal documents
- Quick analysis
- Transparency required
- Known categories

When to Use Machine Learning:

- Informal text (social media)
- Complex classification
- New categories
- Large datasets

When to Use Word Embeddings:

- Capturing meaning
- Semantic relationships
- Large corpus available
- Advanced applications

Key Insight: Choose approach based on your specific needs!

Data Quality:

- Use appropriate data sources
- Clean text carefully
- Document pre-processing steps

Model Selection:

- Start simple (dictionary)
- Progress to complex (ML, embeddings)
- Validate with test data

Evaluation:

- Use multiple metrics
- Consider business context
- Document choices

Regulatory Compliance:

- Ensure transparency
- Document methodology
- Be ready for audit

Key Insight: Rigorous methodology is essential!

Emerging Trends:

1 Large Language Models

- GPT, BERT, LLaMA
- Contextual understanding

2 Multilingual NLP

- Cross-language applications
- Global finance

3 Fine-tuning

- Domain-specific models
- Financial NLP

4 Explainable AI

- Interpretability
- Regulatory compliance

Key Insight: NLP is rapidly evolving!

Applications:

1 Credit Risk

- Analyze financial reports
- Assess management quality

2 Market Risk

- Sentiment analysis
- News impact assessment

3 Operational Risk

- Compliance monitoring
- Fraud detection

4 Regulatory Compliance

- Document review
- Regulatory filings

Key Insight: NLP is critical for modern risk management!

Technical Considerations:

1 Data Pipeline

- Collection
- Storage
- Processing

2 Infrastructure

- Computing resources
- Cloud vs on-premise

3 Maintenance

- Model updates
- Performance monitoring

4 Integration

- API design
- Workflow integration

Key Insight: Implementation requires careful planning!

Key Regulatory Areas:

1 Model Governance

- Documentation
- Validation
- Monitoring

2 Fairness and Bias

- Avoid discrimination
- Equal treatment

3 Transparency

- Explainable decisions
- Audit trail

4 Data Privacy

- GDPR, CCPA
- Data protection

Key Insight: Regulatory compliance is non-negotiable!

Learning Objectives:

- ➊ Discuss applications of NLP
 - Fraud detection, sentiment analysis, document review
- ➋ Describe pre-processing steps
 - Tokenization, stop words, stemming/lemmatization
- ➌ Discuss BoW and n-grams
 - Word counting, context capture
- ➍ Explain dictionary vs ML approaches
 - Advantages and limitations of each
- ➎ Illustrate TF-IDF
 - Word importance weighting
- ➏ Explain Naïve Bayes
 - Classification for documents
- ➐ Discuss evaluation
 - Metrics, speed, data requirements

All objectives covered in this chapter!

NLP Overview:

- NLP enables computers to understand human language
- Pre-processing converts raw text to analyzable format
- Feature extraction creates numeric vectors (BoW, TF-IDF, Embeddings)

Key Techniques:

- Dictionary approach: Simple, transparent, fast
- Naïve Bayes: Simple, fast, generative classifier
- Word Embeddings: Captures meaning and relationships

Evaluation:

- Confusion matrix metrics (Accuracy, Precision, Recall, F1)
- Consider speed and data requirements
- Unlabeled data requires alternative approaches

Financial Applications:

- Fraud detection, sentiment analysis, document review, compliance

Exam Success: Understand the complete pipeline from text to insights!